

反射(Class) => 对所有类进行了抽象

1.反射

反射: 加载类, 反射出类的各个组成部分 构造方法,属性(非静态属性和静态属性),方法(非静态方法, 静态方法)

获取Class的方式

- 类名.class
- 对象.getClass()
- Class.forName("包名.类名");

反射出构造方法,就可以创建对象

如果是私有的成员用getDeclared相关的方法, 并且开启访问权限。

1.1 编写Person

```
public class Person {

    private String name="奥利给";

    public Integer age=22;

    public static String addresss;

    public Person() {

    }

    private Person(String name) {
        this.name=name;
    }

    public Person(Integer age,String name) {
        this.age=age;
        this.name=name;
    }

    public void aa() {
        System.out.println("奥利给");
    }

    private int bb() {
        System.out.println("bb");
        return 100;
    }

    public int sum(int a,int b) {
        return a+b;
    }

    public static void cc(int a) {
```

```

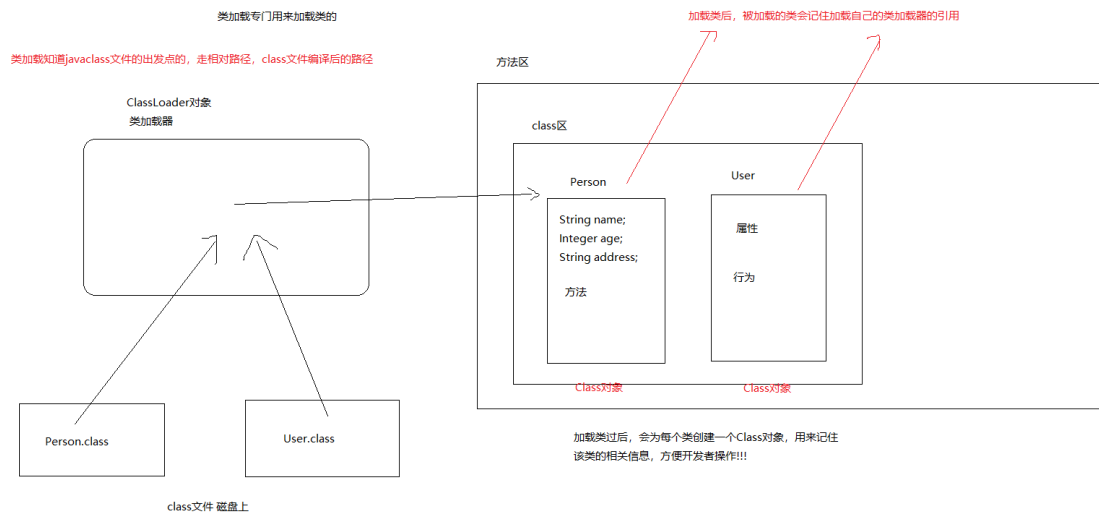
        System.out.println("cc:"+a);
    }

}

```

1.2 加载类

1.2.1 加载类的过程



```

@Test
public void testLoadClass() throws ClassNotFoundException {

    //1. 加载类

    //1. 方式1
    //static Class<?> forName(String className)
    //返回与带有给定字符串名的类或接口相关联的 Class 对象。

    //加载类，并且返回该类的Class对象
    Class clazz=Class.forName("com.gxa.demo.Person");

    System.out.println(clazz);

    //2. 方式2
    Class clazz2=Person.class;
    System.out.println(clazz2);

    //3. 方式三
    Class clazz3=new Person().getClass();
    System.out.println(clazz3);
}

```

```
}
```

1.2.2 获取类加载器

```
//1.如何获得类加载器?  
  
//1.获取加载类后的class对象  
Class clazz=TestPerson.class;  
  
//2.获取类加载器  
ClassLoader classLoader = clazz.getClassLoader();  
  
System.out.println(classLoader);
```

1.2.3 使用类加载器加载其它的文件

```
@Test  
public void test2() {  
  
    //1.如何获得类加载器?  
  
    //1.获取加载类后的class对象  
    Class clazz=TestPerson.class;  
  
    //2.获取类加载器  
    ClassLoader classLoader = clazz.getClassLoader();  
  
    System.out.println(classLoader);  
  
    //3.使用类加载器加载文件  
    InputStream in = classLoader.getResourceAsStream("jdbc.properties");  
  
    System.out.println(in);  
  
}
```

改造JdbcUtils

```
public class JdbcUtils {  
  
    private static Properties config=new Properties();  
  
    //静态代码块 在加载类的时候执行  
    static {
```

```

    try {

        //1.加载配置文件
        //使用类加载器加载properties文件
        Class clazz=JdbcUtils.class;
        ClassLoader classLoader = clazz.getClassLoader();
        InputStream in = classLoader.getResourceAsStream("jdbc.properties");

        //2.把properties文件的资源读取到properties集合中
        config.load(in);

        //3.注册驱动
        Class.forName(config.getProperty("driverClassName"));

    } catch (Exception e) {
        e.printStackTrace();

        //把编译时异常转换成运行异常
        throw new RuntimeException("加载驱动失败!");
    }

}

/**
 * 获取链接
 * @return
 */
public static Connection getConnection() {
    try {

        Connection connection =
            DriverManager.getConnection(config.getProperty("url"),
                                       config.getProperty("username"),
                                       config.getProperty("password"));

        return connection;
    } catch (SQLException e) {
        e.printStackTrace();
        throw new RuntimeException("获取链接失败!");
    }
}

/**
 * 释放资源
 * @param conn
 * @param st
 * @param rs
 */
public static void release(Connection conn, Statement st, ResultSet rs) {
    //关闭资源之前一定要判断
    if(rs!=null) {

        try {

```

```

        rs.close();
    }catch (Exception e) {
        e.printStackTrace();
    }
    //让jvm回收没有被关闭的rs对象
    rs=null;
}

if(st!=null) {

    try {
        st.close();

    }catch (Exception e) {
        e.printStackTrace();
    }
    st=null;
}

if(conn!=null) {

    try {
        conn.close();
    }catch (Exception e) {
        e.printStackTrace();
    }

    conn=null;
}
}
}

```

getDeclared... 获取所有的，包括私有的

1.3 反射构造器

构造器: Constructor

获取构造器
一般都是
get+Constructor+其他
单词(可以有参数);

参数为: 参数的反射
(参数类型.class)

创建对象
Class对象.newInstance(); // 调用无参构造器

Class对象. getConstructor(参数类型.class).newInstance (参数); // 调用有参构造器

构造器.setAccessible(true); 能访问私有构造器 (方法同理)

```

@Test
public void test6() throws Exception {    ***

    //1.加载类
    Class clazz=Person.class;

    //2.反射无参构造器的简写版
    Person person=(Person) clazz.newInstance();

    System.out.println(person);

}

```

```

@Test
public void test3() throws Exception {

    //1.加载类
    Class clazz=Person.class;

    //2.反射无参构造器
    Constructor constructor = clazz.getConstructor(null);

    //3.使用构造器来创建对象newInstance
    Person person = (Person) constructor.newInstance(null);

    System.out.println(person);

}

```

```

@Test
public void test4() throws Exception {

    //1.加载类
    Class clazz=Person.class;

    //private Person(String name)

    //2.反射无参构造器
    //getDeclaredConstructor 获取所有的构造器，不管是私有的还是公有的
    Constructor constructor = clazz.getDeclaredConstructor(String.class);

    //开启访问权限
    constructor.setAccessible(true);

    //3.使用构造器来创建对象newInstance
    Person person = (Person) constructor.newInstance("哈哈");

    System.out.println(person);

}

```

```

    }

    @Test
    public void test5() throws Exception {

        //1.加载类
        Class clazz=Person.class;

        //public Person(Integer age,String name)

        Constructor constructor =
        clazz.getConstructor(Integer.class,String.class);

        Person person = (Person) constructor.newInstance(21,"奥利给");

        System.out.println(person);

    }

    @Test
    public void test6() throws Exception {

        //1.加载类
        Class clazz=Person.class;

        //2.反射无参构造器的简写版
        Person person=(Person) clazz.newInstance();

        System.out.println(person);

    }

```

1.4 反射属性

属性: Field
获取自己写的公有属性 get + Field + 其他单词()
get + Declared + Field + 其他单词() 获取自己写的所有属性

```

@Test
public void test1() throws Exception {

    //    private String name="奥利给";

    Class clazz=Person.class;

    //反射获取对应的属性
    Field f = clazz.getDeclaredField("name");

    f.setAccessible(true);

    Person person=new Person();

```

```

        Object value = f.get(person);

        System.out.println(value);

    }

    @Test
    public void test2() throws Exception {

        Class clazz=Person.class;

        //反射获取对应的属性
        Field f = clazz.getField("age");

        Person person=new Person();

        Object value = f.get(person);

        System.out.println(value);

        f.set(person, 99);

        value = f.get(person);

        System.out.println(value);

    }

    @Test
    public void test3() throws Exception {

        //public static String adderss="火星";

        Class clazz=Person.class;

        //反射获取对应的属性
        Field f = clazz.getField("adderss");

        Object value = f.get(null);

        System.out.println(value);

    }

```


1.5 反射方法

方法: Method

获取所有（包括父类方法）方法 `get + Method + 其他单词` (可以有参数);

参数需要传入Class, 原因: 方法有重载

获取自己写的方法 `get + Declared + Method + ...(参数)`;

调用方法: `Method对象.invoke(该方法的对象, 参数)`

```
public class TestPerson3 {

    // public void aa() {
    //     System.out.println("奥利给");
    // }

    @Test
    public void test1() throws Exception{

        //1.加载类
        Class clazz=Person.class;

        //2.反射方法
        Method method = clazz.getMethod("aa", null);

        //3.执行方法
        method.invoke(new Person(), null);

    }

    // private int bb() {
    //     System.out.println("bb");
    //     return 100;
    // }

    @Test
    public void test2() throws Exception{

        //1.加载类
        Class clazz=Person.class;

        //2.反射方法
        Method method = clazz.getDeclaredMethod("bb", null);

        method.setAccessible(true);

        //3.执行方法
        Object result=method.invoke(new Person(), null);
        System.out.println(result);

    }

    // public int sum(int a,int b) {
    //     return a+b;
    // }

    @Test
    public void test3() throws Exception{

        //1.加载类
        Class clazz=Person.class;
```

```

        //2.反射方法
        Method method = clazz.getMethod("sum", int.class,int.class);

        //3.执行方法
        Object result=method.invoke(new Person(), 1,2);
        System.out.println(result);

    }

    // public static void cc(int a) {
    //     System.out.println("cc:"+a);
    // }

    @Test
    public void test4() throws Exception{

        //1.加载类
        Class clazz=Person.class;

        //2.反射方法
        Method method = clazz.getMethod("cc", int.class);

        //3.执行方法
        method.invoke(null, 1);

    }

}

```

1.6 反射的案例

```

public interface UserDao {

    public void addUser();

}

```

```
public class UserDaoJdbcImpl implements UserDao{

    @Override
    public void addUser() {

        System.out.println("用传统数据库mysql来保存数据!");

    }

}
```

```
public class UserDaoMongdbImpl implements UserDao{

    @Override
    public void addUser() {

        System.out.println("mongodb的实现!");

    }

}
```

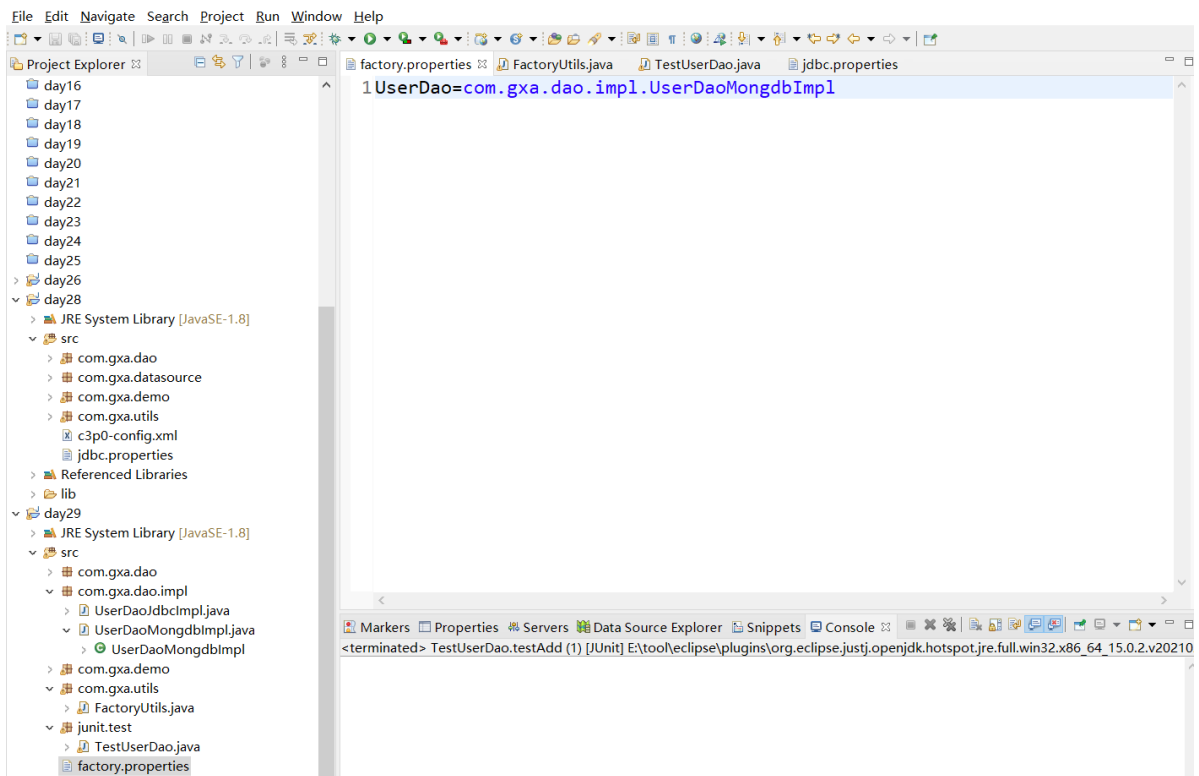
```
@Test
public void testAdd() {

    //UserDao userDao=new UserDaoJdbcImpl();

    UserDao userDao=new UserDaoMongdbImpl();

    userDao.addUser();

}
```



```
public class FactoryUtils {

    private static Properties config=new Properties();

    static {

        try {
            InputStream in =
FactoryUtils.class.getClassLoader().getResourceAsStream("factory.properties");
            config.load(in);
        }catch (Exception e) {
            e.printStackTrace();
            throw new RuntimeException("加载配置文件失败!");
        }
    }

    /**
     * 传递指定的接口名，返回接口对应的而实现类的对象
     * @param name
     * @return
     * @throws ClassNotFoundException
     */
    public static Object getObject(String name) throws Exception {

        //1.根据接口名来获取接口对应的实现类的全限定名
        String classNname=config.getProperty(name);

        //2.加载类
        Class clazz = Class.forName(classNname);

        //3.反射创建对象
```

```
        return clazz.newInstance();  
    }  
}
```

